

MỤC LỤC

MỤC LỤC.....	1
Phần 1: Framework & Xây dựng RESTful API.....	2
1. Lựa chọn Framework Python: Django.....	2
2. Tạo Project & Cấu trúc.....	2
a. Khởi tạo project:.....	2
b. Tìm hiểu cấu trúc thư mục:.....	2
c. Tìm hiểu file cấu hình: settings.py.....	2
3. Xây dựng RESTful API cơ bản.....	3
a. Khái niệm REST API & HTTP Methods:.....	3
b. Cách định nghĩa Routes / Controllers:.....	4
c. Xử lý Request:.....	5
d. Xử lý Response:.....	6
4. Thực hành.....	7
Đề bài:.....	7
Báo cáo:.....	7

Phần 1: Framework & Xây dựng RESTful API

1. Lựa chọn Framework Python: Django

2. Tạo Project & Cấu trúc

a. Khởi tạo project:

`django-admin startproject myproject`

b. Tìm hiểu cấu trúc thư mục:

- **.venv/** Là môi trường ảo của project, chứa bản Python và các thư viện riêng biệt giúp project hoạt động độc lập, không xung đột thư viện với hệ thống.
- **manage.py**: File điều khiển toàn project. Dùng để chạy server, migrate database, ...
- **__init__.py**: Đánh dấu thư mục là một package Python, giúp các file trong thư mục có thể import qua lại.
- **settings.py**: Nơi cấu hình trung tâm của Django: database, app, static files, timezone, middleware...
- **urls.py**: Xác định tuyến đường cho request. Mapping URL đến view tương ứng.
- **wsgi.py**: Cấu hình chạy project trên server WSGI. Dùng cho ứng dụng đồng bộ.
- **asgi.py**: Tương tự wsgi nhưng cho server hỗ trợ bất đồng bộ. Dùng khi triển khai với WebSocket hoặc async server như Uvicorn.

c. Tìm hiểu file cấu hình: settings.py

settings.py là bộ nao của Django project, nơi lưu toàn bộ cấu hình điều khiển ứng dụng. Thường chia file này thành nhiều phần nhỏ như **settings/base.py**, **settings/dev.py**, **settings/prod.py** để tách môi trường phát triển và production.

- Cấu hình cơ bản

- + **BASE_DIR**: đường dẫn gốc của project, giúp định vị các thư mục con.
- + **SECRET_KEY**: chuỗi bảo mật nội bộ, dùng để mã hóa session và token.
- + **DEBUG**: nếu True, Django hiển thị lỗi chi tiết (chỉ bật khi dev).
- + **ALLOWED_HOSTS**: danh sách domain/IP được phép truy cập app.

- Ứng dụng

- + **INSTALLED_APPS**: liệt kê các app được Django kích hoạt

- + **MIDDLEWARE**: danh sách các lớp xử lý request/response trung gian (bảo mật, session, xác thực...).
- + **ROOT_URLCONF**: đường dẫn đến file urls.py chính của project.
- + **WSGI_APPLICATION**: trỏ đến file wsgi.py để deploy WSGI server.
- **Cấu hình cơ sở dữ liệu: DATABASES**: định nghĩa kết nối DB, mặc định là SQLite.
- **Template & Static Files**
 - + **TEMPLATES**: khai báo nơi chứa HTML template, backend render, context processors.
 - + **STATIC_URL, STATICFILES_DIRS**: cấu hình nơi lưu CSS, JS, ảnh tĩnh.
- **Ngôn ngữ & Thời gian**
 - + **LANGUAGE_CODE**: ngôn ngữ mặc định.
 - + **TIME_ZONE**: múi giờ hệ thống.
 - + **USE_I18N, USE_TZ**: bật / tắt đa ngôn ngữ và timezone.
- **Authentication & Session**
 - + **AUTH_PASSWORD_VALIDATORS**: quy định độ mạnh của mật khẩu.
 - + **AUTH_USER_MODEL**: cho phép thay thế model người dùng mặc định.

3. Xây dựng RESTful API cơ bản

a. Khái niệm REST API & HTTP Methods:

REST API (Representational State Transfer API) là chuẩn giao tiếp giữa client và server qua HTTP, giúp các hệ thống trao đổi dữ liệu một cách thống nhất. Mỗi tài nguyên (resource) như *user*, *product*, *order*... được biểu diễn bằng URL, và ta thao tác trên nó bằng HTTP methods.

Các phương thức chính:

- **GET**: Lấy dữ liệu (ví dụ /api/users để lấy danh sách người dùng).
- **POST**: Thêm mới dữ liệu (gửi JSON lên server để tạo user)
- **PUT/PATCH**: Cập nhật dữ liệu (PUT thay toàn bộ, PATCH thay một phần)
- **DELETE**: Xóa dữ liệu (ví dụ /api/users/5 để xóa user có id = 5).

b. Cách định nghĩa Routes / Controllers:

Cấu trúc Project khi có thêm phần App api

Trong `First_project_django.urls.py` có:

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('api.urls')),
]
```

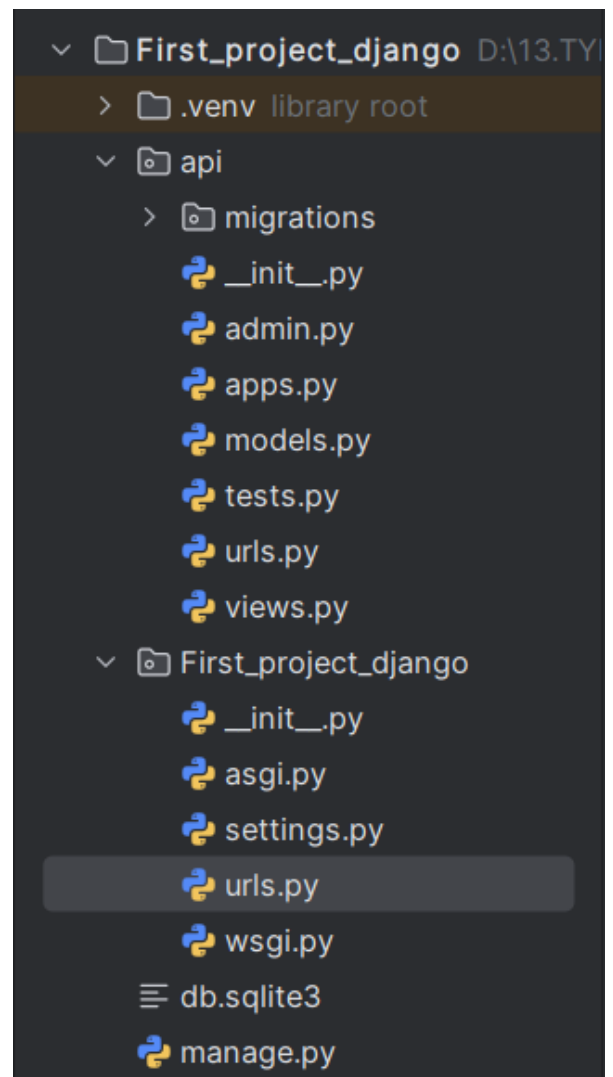
Hoang

Với `path('', include('api.urls'))` rằng hãy trở đến `api.urls`

Trong `api.urls`, có:

```
urlpatterns = [
    path('api/todo', views.todo_list),
    path('api/user', views.user_list),
]
```

Hoang



Định nghĩa URL và là hàm sẽ được gọi khi có request đến URL đó.

Trong đó các hàm trả về Json gì thì được định nghĩa trong `api.views.py`:

```
from django.http import JsonResponse

def todo_list(request):
    data = {"todos": ["Học Django", "Viết REST API"]}
    return JsonResponse(data)

def user_list(request):
    data = {"users": ["Hoàng", "Mèo"]}
    return JsonResponse(data)
```

Hoang

c. Xử lý Request:

- Path Variable (`/api/user/{id}`)

Khi URL chứa biến, ví dụ `/api/user/5`, ta định nghĩa route trong `urls.py`:

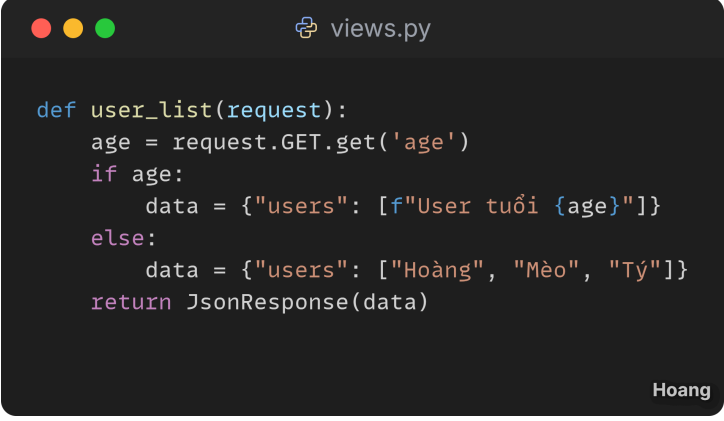
`path('api/user/<int:id>', views.get_user)`

Sau đó định nghĩa hàm `get_user` ở `views.py`

- Query Param (`/api/users?age=20`)

Khi dữ liệu nằm sau dấu `?`, ta đọc bằng `request.GET`:

Ta định nghĩa lại hàm `user_list` trong `views.py`



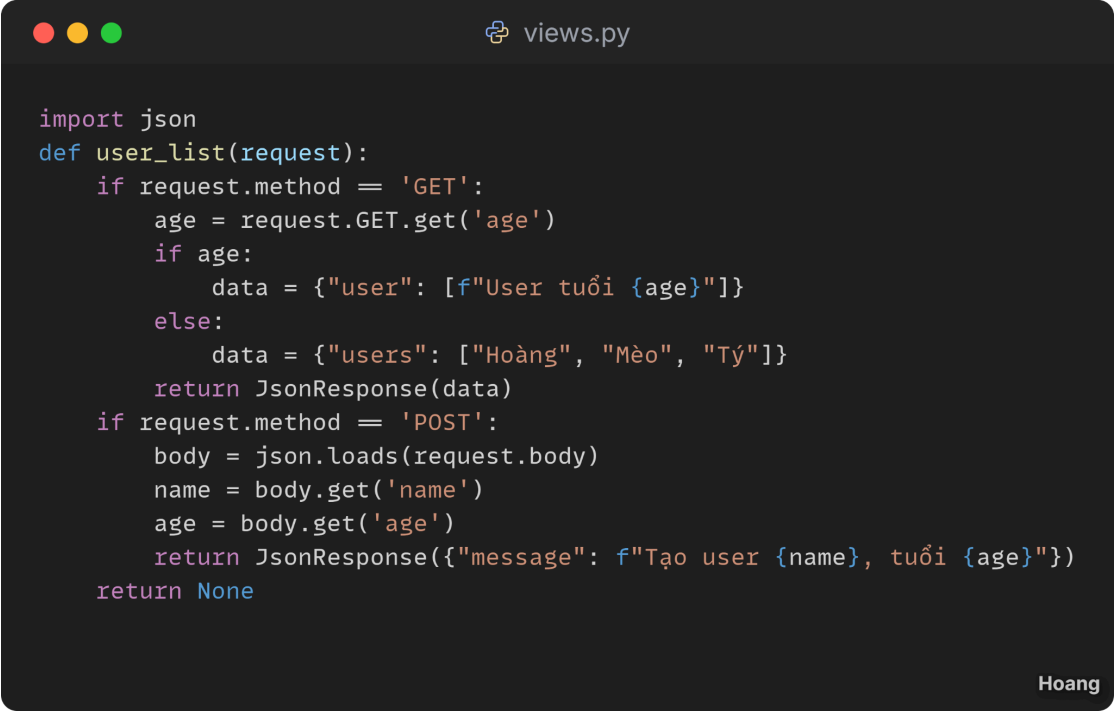
```
def user_list(request):
    age = request.GET.get('age')
    if age:
        data = {"users": [f"User tuổi {age}"]}
    else:
        data = {"users": ["Hoàng", "Mèo", "Tý"]}
    return JsonResponse(data)
```

Hoang

- Request Body (JSON payload)

Khi client gửi dữ liệu bằng **POST, PUT, PATCH**, ta đọc body JSON:

Ta thêm xử lý với phương thức POST trong hàm `user_list` trong `views.py`



```
import json
def user_list(request):
    if request.method == 'GET':
        age = request.GET.get('age')
        if age:
            data = {"user": [f"User tuổi {age}"]}
        else:
            data = {"users": ["Hoàng", "Mèo", "Tý"]}
        return JsonResponse(data)
    if request.method == 'POST':
        body = json.loads(request.body)
        name = body.get('name')
        age = body.get('age')
        return JsonResponse({"message": f"Tạo user {name}, tuổi {age}"})
    return None
```

Hoang

d. Xử lý Response:

- Trả về dữ liệu dạng JSON (như trên)
- Gửi kèm HTTP Status Code thích hợp (200, 201, 400, 404, ...).

Ta cập nhật hàm `user_list` như sau để gửi kèm HTTP Status.

```
views.py

def user_list(request):
    data = {}
    status_code = 200

    if request.method == 'GET':
        age = request.GET.get('age')
        if age:
            data = {"user": [f"User tuổi {age}"]}
        else:
            data = {"users": ["Hoàng", "Mèo", "Tý"]}
        status_code = 200

    elif request.method == 'POST':
        try:
            body = json.loads(request.body)
            name = body.get('name')
            age = body.get('age')
            if not name or not age:
                data = {"error": "Thiếu thông tin name hoặc age"}
                status_code = 400
            else:
                data = {"message": f"Tạo user {name}, tuổi {age}"}
                status_code = 201
        except json.JSONDecodeError:
            data = {"error": "Dữ liệu JSON không hợp lệ"}
            status_code = 400
    else:
        data = {"error": "Phương thức không được hỗ trợ"}
        status_code = 405

    return JsonResponse(data, status=status_code)
```

Hoang

4. Thực hành

Đề bài:

Bài tập: Xây dựng API CRUD quản lý một đối tượng

- GET /api/user → Lấy danh sách
- GET /api/user/{id} → Lấy chi tiết
- POST /api/user → Thêm mới
- PUT /api/user/{id} → Cập nhật
- DELETE /api/user/{id} → Xóa

Dữ liệu:

- Lưu tạm trong List/Map (Java/Python).
- Chưa cần kết nối Database.

Output sau phần 1:

- Trình bày phần Lý thuyết.
- Một project backend chạy được.
- Có ít nhất 5 API CRUD hoạt động và trả về JSON đúng format.
- Quay video demo.
- Kiểm thử API bằng Postman.

Báo cáo:

Source Code em đã để ở github ạ.

Video báo cáo phần 4:

https://youtu.be/U-efdVi_OOU